LL-Parser

BC George (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Wiederholung

PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie könnnen wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

$a^i b^{i+1}$ (,) { , }

Motivation

Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (=Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

Themen für heute

- Automatische Generierung von Top-Down-Parsern aus LL-Grammatiken



if (i == 5) . . .

Syntaxanalyse

Syntax

⇒ Grammatik

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Syntaxanalyse ist die Bestimmung, ob Eingabedaten einer vorgegebenen Syntax entsprechen.

d.h. es existiert ein Ableit...baum

Diese vorgegebene Syntax wird im Compilerbau mit einer kontextfreien Grammatik beschrieben und mit einem sogenannten **Parser** analysiert.

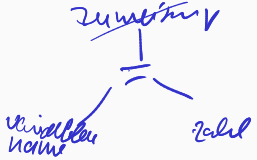
Wir beshäftigen uns heute mit LL-Parsing, mit dem man eine Teilmenge der eindeutigen kontextfreien Grammatiken syntaktich analysieren kann.

Der Ableitungsbaumwird von oben nach unten aufgebaut.

L : von links nach rechts abarbeiten
L : Linksableitung aufbauen
} ⇒ top down

Ziele der Syntaxanalyse

- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- evtl. Fehlerkorrektur
- Bestimmung der syntaktischen Struktur eines Programms
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltablelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche.



Arten d. Syntaxanalyse

- Top-down : Ableitungsbaum von oben aufbauen

  LL

- Bottom-up

  LR

Eine formale Sprache heißt
inhärent mehrdeutig,
wenn jede Grammatik
für diese Sprache mehrdeutig
ist.

Solche Sprachen existieren.

---

nutzloses Nichtterminal A:
es gibt ein Wort w mit $A \overset{*}{\Rightarrow} w$

---

unerreichbar:
es gibt keinen Weg $S \overset{*}{\Rightarrow} \alpha A \beta$

---

ohne das,
reduzierte Grammatik

Algorithmus: (RD)

RecursiveDescent(X, a) // X ∈ N, a ∈ T

for a production $X \to Y_1 Y_2 \dots Y_n$ $\in (N \cup T)$

```
for i = 1 to n
    if Y_i ∈ N
        RecursiveDescent(Y_i, a)
    else
        if Y_i ≠ a
            error
```

X soll abgeleitet werden,
a ist das nächste Token

- nicht deterministisch
- Backtracking

Problemfälle

1) $A \to Aa$ Linksrekursion direkt

$A \to Ba$

$B \to AB$ → Algorithmen 0 anschauen

2) $A \to BC \mid BD$:

Linksfaktorisierung

$A \to BE$

$E \to C \mid D$

Einmal Nichtterminal

LL(k)-Grammatiken

First-Mengen



$S \to A \mid B \mid C$

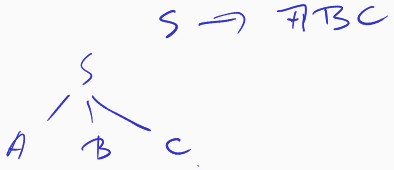
Welche Produktion nehmen?

Wir brauchen die "terminalen k-Anfänge" von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren *First* - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \setminus T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \overset{*}{\Rightarrow} w, \text{mit } w \in T^*, First_k(w) = \{v\}\}$

Linksableitungen



Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \overset{*}{\Rightarrow}_l \beta$.

LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ ist genau dann eine *LL(k)*-Grammatik, wenn für alle Linksableitungen der Form:

$S \overset{*}{\Rightarrow}_l wA\gamma \Rightarrow_l w\alpha\gamma \overset{*}{\Rightarrow}_l wx$

und

$S \overset{*}{\Rightarrow}_l wA\gamma \Rightarrow_l w\beta\gamma \overset{*}{\Rightarrow}_l wy$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$\alpha = \beta$

LL(1)-Grammatiken

$S \to A \mid C$

$A \to aBA$

$B \to bB \mid \varepsilon$

$C \to cc$

$First(S) = \{a, c\}$

$First(A) = \{a\}$

$First(B) = \{b, \varepsilon\}$

$First(C) = \{c\}$

nächstes

LL(k)-Sprachen

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von *LL(k+1)*-Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine *LL(1)* - Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine LL(k)- (meist LL(1)-) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

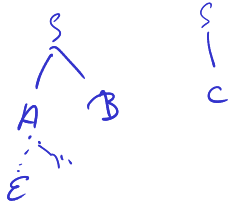
$REG \subsetneq LL(1) \subsetneq LL(2) \ldots$

$S \to AB \mid c \ldots$

$A \to \ldots \mid \varepsilon$

$B \to \ldots \mid b$

In First (S) müssen auch die Symbole enthalten sein, die auf A folgen können



$\Rightarrow$ Follow-Mengen berücksichtigen

$\text{Follow}_k(\beta) = \{ w \in T^+ \mid \exists \alpha, \gamma \in (N \cup T)^* \text{ mit } S \overset{*}{\Rightarrow}_R \alpha \beta \gamma \text{ und } w \in \text{First}_k(\gamma) \}$

First (β): terminalen Anfänge aller Ableitungen von β

Follow (β) " " aller Elemente die hinter β stehen können

LL(k)-Sprachen

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von *LL(k)*-Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von *LL(k+1)*-Grammatiken erzeugten Sprachen.

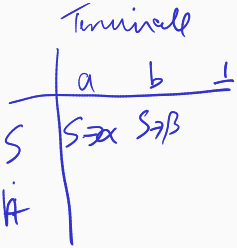
Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine *LL(1)* - Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine LL(k)- (meist LL(1)-) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik G = (N, T, P, S)

Ausgabe: Eine Parsertabelle *P*




```
for each production X → α
   for each a ∈ First(α)
      add X → α to P[X, a]
   if ε ∈ First(α)
      for each b ∈ Follow(α)
         add X → α to P[X, b]
         if ε ∈ First(α) and ⊥ ∈ Follow(X)
            add X → α to P[X, ⊥]
```

Abbildung 1: Algorithmus zur Generierung einer LL-Parsertabelle

Hier ist ⊥ das Endezeichen des Inputs. Statt $First_1(\alpha)$ wird oft nur $First(\alpha)$ geschrieben.

LL-Parsertabellen

$S \to A \perp \mid C a B \perp$

$A \to Bc$

$C \to d$

$B \to b$

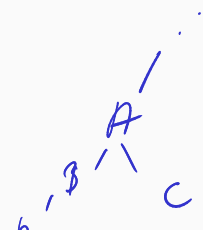
$First(S) = \{d, b\}$

$First(A) = \{b\}$

	a	b	d	⊥
S	—		$S \to A\perp$	$S \to CaB\perp$
A				
B				

In der Tabelle steht, welche Belegung genommen werden muss, wenn das nächste Zeichen über der Tabelle entspricht

Zwei oder Einträge: Grammatik ist nicht LL(1)



LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt. Diesen Stack kann man auch “selbst programmieren”, d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik G = (N, T, P, S), eine Parsertabelle P mit "$w \perp$" als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w, Fehler sonst

```
a = next_token()
X = top of stack  // entfernt X vom Stack

while X ≠ ⊥

   if X = a
      a = next_token()

   else if X ∈ T
      error

   else if P[X, a] leer
      error

   else if P[X, a] = X → Y1Y2 ... Yk
      process_production(X → Y1Y2 ... Yk)
      push(Y1Y2 ... Yk) //Y1 = top of stack

   X = top of stack
```

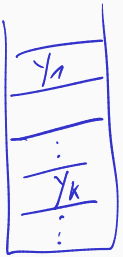
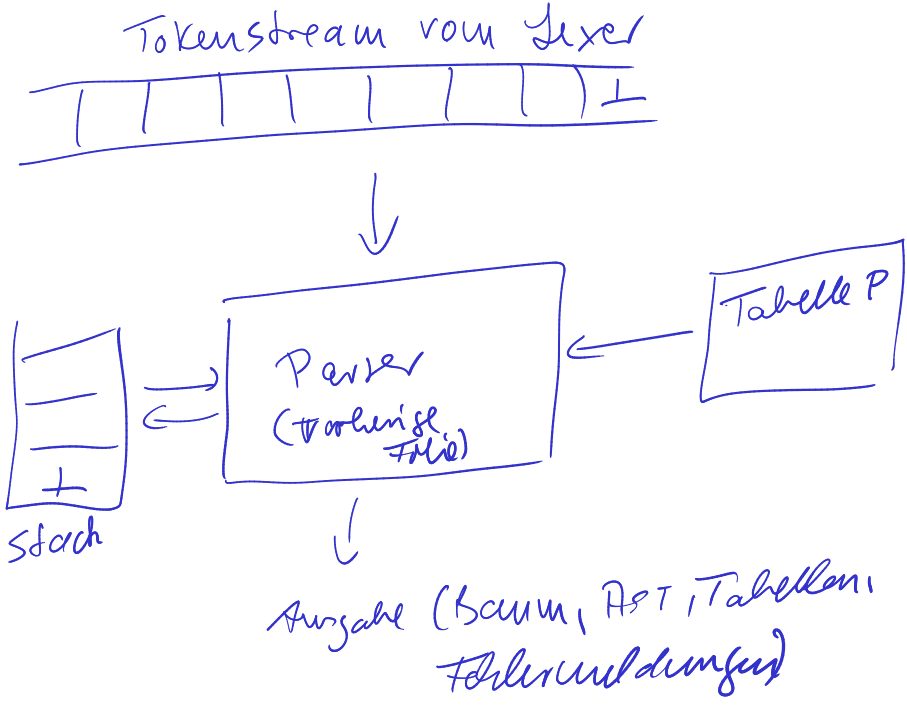


Abbildung 2: Algorithmus zum tabellengesteuerten LL-Parsen


Tokenstream vom Lexer
⊥
Parser
(Tabelle P)
Tabelle P
⊥
Stack
Ausgabe (Baum, AST, Tabellen,
Fehlermeldungen)

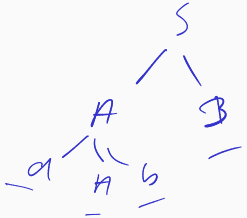
Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Fehlerkorrektur
- Format für die Weiterverarbeitung:
  - Ableitungsbaum oder Syntaxbaum oder Parse Tree
  - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, ...)
- Symboltabelle

Wrap-Up

Wrap-Up

$S \to ATB \mid b$
$A \to aAb \mid \varepsilon$
$B \to bbA \mid b$





- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

$aabbb \in L$?



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *0041a57 2026-09-30 theory: temporarely remove links to old annatated slides*